Walk us through it

Headless CMS Admin Panel — The Agile Monkeys Frontend Challenge 2026
Author: Jhon Ávila · **Date:** 2026-08-16 · **Reading time:** ~12 minutes

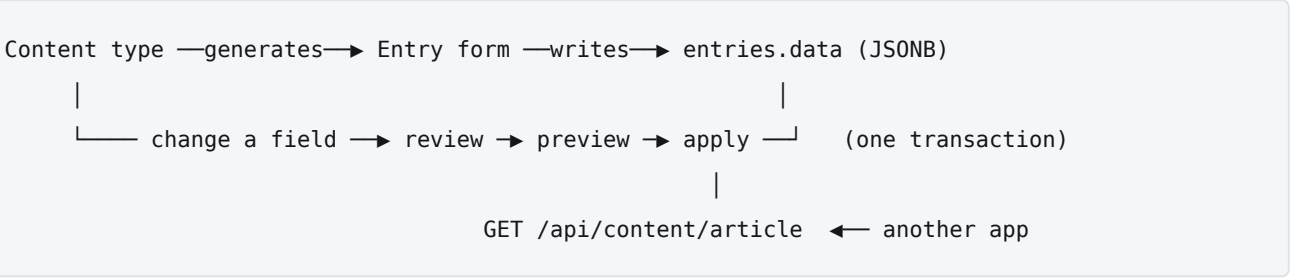
> This is the async walkthrough deliverable. It covers the architecture, the data model, how real-time and schema changes work, the trade-offs I weighed, and what I would do next. Every claim in it is checkable against the code — section 11 says where to look.
>
> **In a hurry?** Sections 1, 2 and 7 are the argument. The rest is evidence.

Companion documents: `PRD.md` (scope and acceptance criteria) · `IMPLEMENTATION_STRATEGY.md` (architecture decisions and their reversals) · `AI_WORKFLOW.md` (how AI was used, and the eleven defects verification caught) · `API.md` · the `MILESTONE_*.md` records.

---

1. The 90-second version

A content modeller defines a content type — `Article`, `Person`, `Product` — through the UI. The entry editor for that type is **generated from the definition**: there is no per-type form code anywhere in the repository. Every open client stays in sync over a websocket. The content is readable over HTTP so another app could consume it. And when a field is renamed, deleted, retyped, made required or re-pointed, the save turns into a **review**: how many entries are affected, what each value becomes, which ones will not convert, and what to do about those — before anything is written.



Stack: Next.js 16 (App Router) · React 19 · TypeScript · Tailwind CSS v4 · Supabase Postgres + Supabase Realtime · Zod v4 compiled at runtime. 237 tests. Typecheck, lint and production build clean.

The brief said *"use any stack you like, keep the backend thin."* The backend here is three tables, five indexes, one trigger function and **one** stored procedure. Everything else is application code.

---

2. The decision everything else follows from

An entry stores its values in a single JSONB column, keyed by field key.

```
create table public.entries (
  id         uuid primary key default gen_random_uuid(),
  schema_id  uuid not null references public.schemas(id) on delete cascade,
  data       jsonb not null default '{}'::jsonb,
  invalid    boolean not null default false,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

The alternative — a real table per content type, with real columns — is more honest about types and gives you foreign keys and per-column constraints for free. I did not take it, for one reason:

> **It turns schema evolution into DDL.** Renaming a field becomes `ALTER TABLE … RENAME COLUMN`. Retyping becomes `ALTER TABLE … ALTER COLUMN TYPE … USING …`, where the `USING` clause is a cast the user cannot see before it runs. You cannot show a per-row preview of a cast. You cannot let someone fix three bad rows and keep the rest.

With JSONB, a schema change is a **data transform** — an ordinary function from old rows to new rows. That function can be run twice: once to render the preview, once to produce the write. That property is the whole feature the challenge asked for, and it is why the trade-off was worth it.

What it costs, stated plainly:

- The database enforces no per-field types or requiredness. Validation lives in a Zod schema compiled at runtime, on the server, on every write.
- A reference field cannot be a foreign key. `validateReferences` in `lib/actions/entries.ts` is the compensating check, and it is a check, not a guarantee — a concurrent delete can still slip past it.
- Sorting by a field sorts JSONB text (`data->>key`). `?order=price` on a numeric field orders lexicographically. Known, documented in `API.md`, not fixed.

Everything from section 4 onwards is downstream of this paragraph.

---

3. Architecture

The layer that matters: pure core, thin shell

The consistent rule across the codebase is that **every decision is a pure function and every side effect is a thin wrapper around one.**

PURE MODULE	DECIDES	SHELL THAT USES IT
`lib/schema/diff.ts`	what changed, and how risky	schema editor, migration review
`lib/migrations/transform.ts`	what each stored value becomes	preview **and** the write
`lib/migrations/analyze.ts`	counts, impacts, preview rows, the write plan	migration review
`lib/schema/zod-builder.ts`	whether a submitted value is valid	entry form, server actions
`lib/realtime/sync-policy.ts`	connection state, refetch, conflict class	realtime provider
`lib/theme/preference.ts`	which theme applies	theme store, pre-paint script

That is why 237 tests cover the interesting behaviour without a single mounted component or live database: the interesting behaviour is not in the components or the database. `lib/api/data.ts` exists purely so the route handlers can be exercised against a stub — the routes have 39 tests and never touch Postgres.

Request paths

There are exactly three ways data moves, and they are deliberately different:

1. **Reads for the UI** — React Server Components call `lib/queries.ts` directly. No client fetch layer, no cache layer. One deliberate exception: the migration review is a client component that calls the analysis server action on mount and again on every resolution change, because its result depends on choices the user has not made yet.
2. **Writes** — Server Actions in `lib/actions/`. Every one re-derives its validation from the *database's current* `fields` rows, never from what the client sent. A stale client cannot smuggle a value in.
3. **Reads for other apps** — Route Handlers under `app/api/content/`. Public, CORS-open, no auth, `Cache-Control: no-store`.

Real-time is a fourth path only in appearance: an event never carries data into the app. It triggers `router.refresh()`, and the server re-renders. More on that in section 6.

What I did *not* build

No component library (dropped shadcn/ui after starting with it in the plan — the surface needed was nine small primitives, and native `<select>`, `<dialog>` and `<input type="date">` give keyboard and screen-reader behaviour for free). No form library. No state manager. No API client layer. Each of those would have been a dependency earning less than its weight at this size.

React Hook Form, its resolver package and TanStack Query were nevertheless sitting in `package.json` from the original plan, imported by nothing. Writing this section is what surfaced them; they are gone now.

---

4. Data model

Three tables. `0001_init.sql` is 167 lines including comments.

```
schemas ──1:N──▶ fields ──self-FK──▶ schemas   (reference targets)
   │
   └────1:N──▶ entries.data (jsonb)
```

`schemas` — `api_id` is unique and format-checked (`^[a-z][a-z0-9_]*$`). It is the public API path, so it is **immutable after creation**: `updateSchemaMeta` deliberately refuses to change it, because renaming it would silently break every consumer. That is a decision, not an oversight, and the code says so.

`fields` — `unique (schema_id, key)`, key format-checked, and a constraint that ties the two together:

```
constraint fields_reference_target check (
  (type =  'reference' and target_schema_id is not null) or
  (type <> 'reference' and target_schema_id is null)
)
```

Note the asymmetry in the two foreign keys: deleting a schema **cascades** to its own fields, but a schema that is someone else's reference target is `on delete restrict`. You can delete a type; you cannot delete a type out from under a type that points at it. The UI checks this first and names the offending content types rather than surfacing a Postgres error.

`entries` — plus `invalid boolean`, set by a migration when a row no longer fits its schema, and cleared automatically the next time the row is saved through the form.

Indexes worth mentioning: a GIN index on `entries.data` (search), a composite `(schema_id, updated_at desc)` (the default list order), and a **partial** index `on entries (schema_id) where invalid`, added for a "what needs attention" view. Being honest about it: that view does not exist yet, so the index is currently unused. Either build the query or drop the index — see §10.

`updated_at` does double duty: it is the optimistic-concurrency token. There is no separate version column.

Security posture, stated once and honestly: there is no authentication — the challenge scopes it out — so RLS is enabled on all three tables with a policy named `"public access"` that is `using (true) with check (true)`, which leaves the key Supabase hands to the browser with full read/write. (The table grants themselves are Supabase's platform defaults, not something this migration sets.) This is a single-tenant demo posture and the migration file says so in a comment. Adding auth would mean replacing those three policies and moving writes behind a session, not restructuring anything.

---

5. The generated editor

No form is written per content type. `components/entry/entry-form.tsx` walks `schema.fields`, calls `rendererFor(field.type)`, and renders whatever comes back.

Validation is compiled **at runtime, from database rows**:

```
const VALIDATORS: Record<FieldType, (field: Field) => z.ZodTypeAny> = {
  text:      () => z.string().min(1, "This field cannot be empty"),
  number:    () => z.number({ message: "Enter a number" }).finite("Enter a finite number"),
  boolean:   () => z.boolean(),
  date:      () => z.string().regex(ISO_DATE, "Enter a date as YYYY-MM-DD")
                    .refine(isRealCalendarDate, { message: "That is not a real date" }),
  reference: () => z.string().uuid("Pick an entry"),
};
```

Two details in there are load-bearing:

- **`isRealCalendarDate` exists because `Date.parse("2026-02-31")` does not fail** — it rolls over to 3 March. A regex-shaped date validator that trusts `Date.parse` accepts 31 February and stores it. The check round-trips the parsed value back through `Date.UTC` and compares.
- The object is closed with `.strip()`, not `.strict()`. Unknown keys are **dropped, not rejected**, so a client running a stale schema cannot smuggle values into storage — but it also does not get an unhelpful error for a field that was deleted while its tab was open.

The extension point the PRD asked for (B5): adding a sixth field type requires **no per-schema work** — that is the property that matters, and it holds. But the honest count of files is six, not the two an in-code comment claims: `FIELD_TYPES` in `types/cms.ts`, `VALIDATORS` here, the renderer registry in `components/fields`, the labels/hints maps and the risk matrix in `lib/schema/constants.ts` and `lib/schema/diff.ts`, the `convertValue` switch in `lib/migrations/transform.ts`, and the `field_type` enum in SQL.

Four of those are exhaustive `Record<FieldType, …>` maps, so **the build breaks until they are all filled in** — the compiler enumerates the work rather than letting a half-added type ship. That is the design I would defend. The comment claiming "two places" is the thing that is wrong, and I found it writing this document.

---

6. Real-time

One rule

> **Events say what changed. The server says what it now is.**

An event never patches client state from its payload. It triggers `router.refresh()`, which refetches the Server Components. That is chattier than applying diffs locally, and it removes the entire class of bug where a client's copy silently drifts from the database. `router.refresh()` preserves client state, so an open form keeps its values while the data behind it updates.

One channel (`"cms-admin"`), three `postgres_changes` listeners — `schemas`, `fields`, `entries`. All three tables are `replica identity full` so DELETE events carry the old row.

The connection state machine

Four states, and the distinctions are deliberate:

SUPABASE-JS STATUS	→	WHY
`SUBSCRIBED`	`live`	
`CHANNEL_ERROR` / `TIMED_OUT`	`connecting` **if** it was already connecting, else `reconnecting`	A cold start that fails is not a lost connection
`CLOSED`	`offline`	
anything else	unchanged	

`reconnecting` is separate from `offline` because telling someone they are disconnected while the client is mid-retry is alarmist and wrong. On any transition **into** `live` from `reconnecting` or `offline`, the client does a full refetch rather than replaying missed events — during a gap the database is the only thing that knows the truth.

Browser `online`/`offline` events feed the same machine, because a websocket can stay nominally open while the network is gone.

Refetches are debounced at **250 ms**. A schema migration touches every entry of a type; without the debounce that is one refetch per row.

Concurrent edits — and the bug that nearly ate them

Optimistic concurrency: the form sends the `updated_at` it loaded, the server compares, and a mismatch returns `conflict: true` instead of writing.

The subtle part is which `updated_at` the form sends:

```
// components/entry/entry-form.tsx
const [baselineUpdatedAt, setBaselineUpdatedAt] = useState(entry?.updated_at ?? "");
```

That is `useState` — deliberately **not** derived from the prop. It moves only when *this* form saves successfully (adopting the token the write returned, so its own echo is not misread as someone else's edit); a re-render caused by anyone else never touches it. Here is why. Real-time calls `router.refresh()`. That re-renders this page with a *new* `entry.updated_at`. If the token tracked the prop, then a colleague saving the same entry would push their timestamp into my form, my next save would match, and I would silently overwrite their work **while the UI reported success**. Freezing the baseline at mount is what keeps the conflict detectable, and updating it only on its own successful write is what stops the form conflicting with itself.

No test would have caught that. It came from asking *"what does the feature I just built do to the feature I built two milestones ago?"* — which is a question about the system, not about the diff in front of you. It is the one I would raise in a pairing session.

When a conflict is detected the user sees an "Edit conflict" notice with an explicit **Overwrite anyway** button, wired to a *separate* server action (`forceUpdateEntry`) rather than a retry flag — an overwrite should always be a second deliberate decision, never a silent retry.

A related classifier, `classifyOpenEntryEvent`, decides whether an incoming event concerns the entry you have open: it filters by row id, distinguishes delete from update, and suppresses **your own echo** by comparing the incoming `updated_at` against the one you already know. Two banners come out of it: *this entry was deleted*, and *changed by someone else* (with **Load their version** / **Keep editing**). A third comes from a separate branch on `isSchemaChange` — *the content type changed* — and that one forces a full reload, because a generated form has to be rebuilt, not refreshed.

---

7. Schema evolution — the core

The brief asked to *"show how you communicate the risk, surface the affected entries, preview before applying, and let people fix data that no longer fits."* Four verbs. Here is each one.

The flow

```
draft fields
   │
   ├─▶ diffFields(saved, draft)          → what changed + risk        (communicate)
   │
   ├─▶ gateChanges(changes, entryCount)  → straight save, or review?
   │
   └─▶ analyzeMigration(...)             → counts, per-field impacts,  (surface)
         │                                    before/after preview rows  (preview)
         │
         ├─▶ user picks a resolution per problem field                 (fix)
         │      convert · default · clear · flag
         │
         └─▶ buildMigrationPlan(...)    → fields[], entries[], deletes
                   │
                   └─▶ rpc apply_schema_migration(...)                 (apply, atomically)
```

Communicate the risk

`diffFields` produces **nine change kinds** — add, delete, rename key, rename label, retype, set required, unset required, retarget reference, reorder — each classified as **safe**, **lossy** or **blocking**.

The single most important line in that file is that fields are matched **by `id`, never by key**. That is what makes a rename distinguishable from a delete-plus-create, and therefore what makes D6 (*rename preserves data*) possible at all. Match by key and every rename is silently a data loss.

Risk is also *contextual*: adding a required field is `safe` on an empty type and `blocking` the moment entries exist. `gateChanges` re-derives that, which is why creating a type is not encumbered by a review flow that has nothing to review.

`date → number` is classified **lossy**, not safe, and this is the one I argued with myself about. Epoch milliseconds are perfectly recoverable, so it's "lossless" in the information-theory sense. But `1843-10-01` becoming `-3984163200000` is not what an editor means by "no data lost". The bytes survive; the meaning does not. So it asks.

Surface the affected entries, and preview

`analyzeMigration` runs the real transform over **every** entry. Each field gets an exact affected count and problem count; changed and flagged totals are reported for the migration as a whole. It deliberately does not sample: "about 40 entries" is not a number you can act on. Only the *preview rows* are capped, at 25 per field, with an explicit `truncated` flag so the UI can say so rather than implying it showed everything.

Each row is a genuine before/after: `"42" → 42`, `"abc" → unconvertible ("not a number")`.

Let people fix what does not fit

Four strategies per problem field: **convert** (accept the default behaviour), **default** (supply a value for rows that fail), **clear** (empty them), or **flag** (write the row and mark it `invalid` for a human).

The nuance worth pointing at: *clear* can resolve "this value will not convert", but it **cannot** resolve "this field is now required and this row is empty" — emptying a value does not satisfy a requirement. The transform distinguishes those two problem kinds and refuses to mark the second resolved. The review UI, however, still offers all four strategies for either kind, so choosing *clear* on a required-empty field silently does nothing useful. The engine is right and the affordance is not; that is a rough edge, listed in §10. Rows written with an unresolved problem carry `invalid = true`, and that flag is recomputed from scratch on every migration, so a later change that fixes a row clears a flag an earlier one set.

Apply atomically

One RPC, one plpgsql function, therefore one transaction: it all lands or none of it does.

Three things in that function are worth a reviewer's attention:

1. The unique constraint has to be deferrable. Swapping two field keys — `body → content` while `content → body` — is legal at the end of the statement batch but violates `unique (schema_id, key)` half way through it. The migration converts the constraint to `deferrable initially immediate` and the function defers it for the duration. It also sets it back to `immediate` *before returning*, so a violation surfaces as an error from this function rather than at `COMMIT` where the caller cannot attribute it.

2. It refuses rather than guesses. If the field list it receives does not account for every field currently on the schema, it raises. That is what happens when someone edits the type in another tab while you are reviewing — The function itself raises `'The field list is incomplete for content type %'`; the app translates that into the sentence a person can act on — *"The schema changed while you were reviewing. Reload and start the review again."*

3. It computes no entry values. It does generate ids for new fields and stamp `updated_at`, but every value that lands in `entries.data` was computed in TypeScript and passed in as JSONB. The tempting design is to do the conversion in PL/pgSQL — it is right there next to the data. That means two implementations of the same rules, and any drift between them produces **a preview that lies**, which is worse than having no preview. So the transform stays in one module, the preview and the write both call it, and a test asserts that property directly:

> `lib/migrations/__tests__/transform.test.ts → describe("preview and apply agree")`

The `comment on function` in the SQL says the same thing, so nobody re-implements it there later.

What only writes what changed

`buildMigrationPlan` includes an entry in the write set **only if its data actually changed** — compared key-by-key, not by `JSON.stringify`. That matters more than it sounds: `JSON.stringify` equality is key-order sensitive, so without it, reordering fields in the UI would have rewritten every row, bumped every `updated_at`, and woken every connected client for a purely cosmetic change. There is a test for exactly that.

---

8. Read API

The brief said *"nothing production-grade, just proof the admin panel is managing real content another app could consume."* So: three endpoints, no auth, `Cache-Control: no-store`, CORS open.

```
curl "http://localhost:3000/api/content"                        # discovery
curl "http://localhost:3000/api/content/article?limit=10"       # collection
curl "http://localhost:3000/api/content/article?expand=author"  # + expansion
curl "http://localhost:3000/api/content/article/<uuid>"         # single entry
```

The response shape is **flat** — field values sit next to `id`, `created_at`, `updated_at`, `invalid` — because that is what a consumer wants to destructure. That is only safe because `RESERVED_FIELD_KEYS` blocks a field named `id` or `created_at` at schema-creation time. The reserved list and the flat shape are the same decision; separating them would be a bug waiting to happen.

The shape follows **the schema, not the row**: a field added after an entry was written reads `null` rather than being absent, and an orphaned JSONB key left by an old schema is dropped. Consumers get a stable shape.

Three decisions I would defend:

- **`?limit=abc` is a 400, not a silent default.** `parseInt` would read that as 12 from `"12abc"`; the parser requires `^-?\d+$`. All invalid parameters are collected and returned together, not just the first.
- **Expansion is one level deep and does not N+1.** Ids across the whole page are collected into a `Set` and resolved in a single batch — one call into the data layer regardless of page size, rather than one per referencing row. Depth is capped because `Person.manager → Person` needs cycle detection otherwise, and that is not what this endpoint is for. An expanded reference whose target is gone becomes `null` rather than vanishing, so the breakage is visible.
- **Status codes distinguish "you are wrong" from "we are down."** An unreachable database is `503`, not `500`. Both of those, plus the fact that internal error text was being echoed to consumers verbatim (`TypeError: fetch failed`), were found by curling the running server — the unit tests were green throughout.

---

9. Trade-offs

The ones with a real cost, and what I gave up.

DECISION	BOUGHT	COST
JSONB entry storage	Schema evolution as a previewable data transform	No DB-level types, requiredness or reference integrity; JSONB-text sorting
Transform in TypeScript, not PL/pgSQL	Preview and write cannot disagree	The migration is not a pure SQL artefact; entries round-trip through the app
Refetch on event, never patch	No client/server drift; open forms keep their values	Chattier than diffing; needs the 250 ms debounce
`updated_at` as the concurrency token	No extra column, no version bookkeeping	Any write bumps it, so a no-op save would false-positive — hence "only write what changed"
Server Actions instead of an API layer for writes	No client fetch code, no duplicated validation	Writes are not callable by an external app; the public API is read-only by construction
No component library	Nine primitives, native a11y, no Radix tree	Everything is hand-rolled, including the focus and disabled states
Semantic colour tokens everywhere	A dark theme cost zero component edits	Enforced only by convention — the audit found six literal colours that had crept in
Two matrices: advisory risk vs. executable conversion	The UI can warn conservatively before running anything	They are separate implementations and *can* drift. See below
No auth, permissive RLS	Scoped out by the brief; kept the timebox on evolution	Not deployable as-is to anything public

The one I would flag in review

`lib/schema/diff.ts` holds a **conversion-risk matrix** used to label changes in the UI, and `lib/migrations/transform.ts` holds the **actual conversion**. They are two implementations of related-but-not-identical rules, and they can disagree at the margins — `text → reference` is labelled *lossy* by the former, but the latter will happily convert a well-formed UUID string.

That direction of disagreement is safe (warn more than you break) and it is why the risk matrix is deliberately the pessimistic one. But nothing *enforces* that direction. The invariant that is strictly enforced is the narrower and more important one: preview and apply share `transformEntry`, and the SQL never re-derives a value. If I were continuing, the first thing I would add is a property test asserting the advisory matrix is never more optimistic than the executable one.

10. What I would improve next

Ordered by what I would actually do first.

1. Close the two-matrix gap. The property test above. It is an hour of work and it removes the only place where two sources of truth about the same question still exist.

2. Test the write path. `lib/actions/*` has no unit tests — the pure logic it calls is thoroughly covered, but the orchestration is not. That is the largest honest gap in the suite, and milestones 1 and 2 both name the server actions as unverified rather than implying coverage that does not exist. Integration tests against a real Postgres are the right shape here, not mocks.

3. Clear the three rough edges writing this document surfaced. Each is small, and each is the kind of thing that only shows up when you have to defend a claim out loud rather than read the code:

- The review offers *clear* as a resolution for a required-empty problem, where it cannot help. Filter the strategies by `ProblemKind`.
- The partial index on `entries (schema_id) where invalid` has no query behind it. Build the "needs attention" view it was meant for, or drop it.
- Three unused dependencies were still declared, and one code comment undercounted the work of adding a field type. Both fixed while writing this; both are the argument for making someone explain a codebase periodically.

4. Make migrations scale past one page. `applySchemaMigration` refuses outright above 5,000 entries rather than silently migrating a prefix. Refusing is correct; stopping there is not. The fix is batching the transform with a cursor and applying in chunks inside the same transaction — the analysis already computes exact counts, so the UI would not need to lie about progress.

5. Undo. Right now a migration is atomic but irreversible. Storing the plan's inverse — the before-values are already computed for the preview — would make "undo the last schema change" a matter of replaying the same RPC with the values swapped. The data to do it is already in hand at apply time and currently thrown away.

6. Real reference integrity. Today deleting a referenced entry marks referrers `invalid`, which is visible but reactive. A `references` join table maintained alongside the JSONB would let the database answer "what points at this?" without a `data->>key` scan, and would make the check a guarantee rather than a race.

7. Field-level locking for the review flow. Two people reviewing migrations on the same type currently resolve by the "field list is incomplete" refusal — correct, but the loser loses their resolutions. An advisory lock held for the duration of a review would be kinder.

8. Sorting that respects field types. Generated expression indexes per numeric/date field, or a typed sidecar column. Today `?order=price` is a string sort and the docs admit it.

9. Then, and only then, the product features — publishing states, drafts, i18n, media. All explicitly out of scope here, and all of them are easier once the migration engine can be undone.

---

11. How to check any of this

Nothing above requires taking my word for it.

CLAIM	WHERE
Change classification and the by-id match	`lib/schema/diff.ts`, `lib/schema/__tests__/diff.test.ts` (31 tests)
The conversion matrix and the preview/apply invariant	`lib/migrations/transform.ts`, `transform.test.ts` (40 tests) — see `describe("preview and apply agree")`
Atomic apply, deferrable constraint, refuse-don't-guess	`supabase/migrations/0002_apply_schema_migration.sql`
Connection state machine, echo suppression	`lib/realtime/sync-policy.ts`, `sync-policy.test.ts` (23 tests)
The frozen concurrency token	`components/entry/entry-form.tsx`, and the comment above it
API status codes and error shielding	`lib/api/errors.ts`, `lib/api/handle-error.ts`, `routes.test.ts` (39 tests)
Runtime validation, the 31-February trap	`lib/schema/zod-builder.ts`, `zod-builder.test.ts` (35 tests)

```
npm test        # 237 tests, 8 files
npm run lint    # clean
npm run build   # clean, 12 routes
```

Each `docs/MILESTONE_*.md` has a **Verification** section that lists what was checked *and what was not*. Milestones 1 through 5 each name something left unverified. The method throughout was to execute rather than read: real Postgres for the SQL, `curl` for the API, headless Chromium for the UI, and a hand-built Phoenix-protocol websocket stub for the real-time layer, since no live Supabase project was available at build time.

---

12. In one paragraph

I chose a storage model that trades database-enforced typing for the ability to treat a schema change as an ordinary, previewable function over data — and then spent the rest of the build making that function the single source of truth, so that what the review screen promises and what the transaction writes cannot come apart. Real-time is deliberately dumb: events say *something changed*, the server says *what it is now*, and the one place that had to be clever about it — the concurrency token — is the place a plausible implementation silently loses a colleague's work. The parts I would do next are not features; they are closing the two remaining gaps between what the code proves and what it merely arranges to be true.